Defining Beans, Component Scanning, and Bean Annotations

Spring Beans and Application Context

- When a Spring application starts, the **ApplicationContext** instantiates objects called **beans** to make them available throughout the Spring ecosystem.
- Beans are **objects managed by Spring**, which can be injected wherever needed.

Contributing Beans to the Context

We can define beans in multiple ways:

Using `@Component`

- The simplest way is **annotating a class with `@Component`**.
- Spring will **scan and automatically instantiate** these classes as beans.

```
@Component
public class ProductRepository {
    // ...
}
```

Using `@Configuration` and `@Bean`

- A **configuration class** can manually define beans.
- `@Configuration` tells Spring that the class **contains bean definitions**.

```
@Configuration
public class PersistenceConfig {

    @Bean
    public ProductRepository productRepository() {
        return new ProductRepository();
    }
}
```

- By default, **Spring Boot scans all classes annotated with `@Component` or `@Configuration`** in the **same package as the main class** and its sub-packages.

Stereotype Annotations

- Stereotype annotations **extend `@Component`** and provide **semantic meaning**.

```
@Repository
public class ProductRepository { ... }

@Service
public class ProductService { ... }

@RestController
@RequestMapping("/products")
public class ProductController { ... }
```

Bean Lifecycle

- The lifecycle of a Spring bean consists of **three phases**:
    1. Initialization
    2. Use
    3. Destroy
- We usually focus on **initialization and destroy phases**, especially for DI.

```
@Component
public class BeanA {
    private static final Logger log = LoggerFactory.getLogger(BeanA.class);

    @PreDestroy
    public void preDestroy() {
        log.info("@PreDestroy method is called.");
    }

    @PostConstruct
    public void postConstruct() {
        log.info("@PostConstruct method is called.");
    }
}
```

Bean Scopes

- Define **how beans are created and managed** in Spring:

```
@Configuration
public class AppConfig {

    @Bean(initMethod="initialize", destroyMethod="destroy")
    @Scope("singleton")
    public BeanB beanB() {
        return new BeanB();
    }

    @Bean(initMethod="initialize", destroyMethod="destroy")
    @Scope("prototype")
    public BeanC beanC() {
        return new BeanC();
    }
}
```

Singleton Scope (Default)

- Spring creates **one instance per container**
- Shared across all parts of the application
- **Eagerly initialized by default**
- Lives throughout the lifecycle of the container

Prototype Scope

- Spring creates a **new instance every time the bean is requested**
- Lifecycle is short-lived
- Spring **does not manage destruction** of prototype beans

Key Takeaways

- **@Component** → simple auto-detected bean
- **@Service / @Repository / @RestController** → add semantic meaning
- **@Configuration + @Bean** → programmatic bean definition
- **Singleton vs Prototype** → control how beans are shared or recreated

Resources

- Spring Bean Scopes
- Spring Component Scanning
- @Component vs @Repository and @Service
- Spring Bean Annotations